Integración por Monte Carlo en memoria compartida con OpenMP

Integrante:

David Steven Anaya González - 2211714

Introducción a la Programación Paralela

Universidad Industrial de Santander

2026-2

1. Decisiones de diseño

Conservé la estructura del programa secuencial: un único ciclo sobre las N muestras que acumula aciertos en dentro. La versión paralela solo cambia cómo se reparte ese ciclo. Usé #pragma omp parallel for reduction(+:dentro) schedule(static). La reducción da a cada hilo un contador privado y los suma al final, así no hay una variable compartida que actualizar en cada iteración. Elegí static porque todas las iteraciones cuestan casi lo mismo, y los datos lo confirmaron: con $N = 10^8$ y 4 hilos, static tardó 0,122071 s de mediana y dynamic 2,111579 s, unas 17,3 veces más. dynamic sin tamaño de bloque asigna una iteración a la vez y paga esa asignación 10^8 veces. El número de hilos se recibe como argumento (argv[2]).

La decisión más importante fue el manejo de las semillas. El enunciado pedía flujos independientes, sin solapamiento y reproducibles para cualquier p. En lugar de dar una semilla distinta a cada hilo con rand_r, usé un generador basado en contador: cada muestra se calcula solo a partir de su índice i y de la semilla base 2211714, con una mezcla tipo splitmix64 y sin estado interno. Así el valor de la muestra i es el mismo sin importar qué hilo la procese. Los flujos no se solapan porque cada índice es único, y el resultado no depende de p ni de la planificación.

2. Problemas que aparecieron y cómo los resolví

- *Errores de compilación en la versión secuencial.* Al compilar montecarlo_secuencial.c el código estaba incompleto: faltaban las bibliotecas, la función main() y la medición con omp_get_wtime(). Los mensajes del compilador me lo mostraron, completé el programa y lo compilé y ejecuté en WSL.
- *Riesgo de estado compartido en rand().* Analicé dos fallos posibles. El primero es una condición de carrera sobre el estado interno, que daría resultados distintos en cada ejecución. El segundo es inicializar todos los hilos con la misma semilla, que repetiría muestras y dejaría menos de N observaciones independientes, con un error aparente engañoso. Eso me llevó al generador por contador.
- *Condición de carrera sobre el acumulador.* Sin reduction(+:dentro), varios hilos leerían y escribirían dentro++ a la vez, se perderían incrementos y la estimación quedaría baja. Por eso el acumulador va en la cláusula.
- *Política de planificación.* El 2,11 s de dynamic frente a 0,12 s de static mostró que una cláusula puede costar más que todo el cálculo.

3. Validación

Validé dos cosas por separado.

Exactitud de π. Con $N = 10^8$ el secuencial dio 3,1415394 en las cinco corridas, con 78 538 485 aciertos. El error absoluto fue $5{,}3 \cdot 10^{-5}$. El error típico esperado es $\sigma = 4 \cdot \sqrt{(p(1-p)/N)} \approx 1{,}6 \cdot 10^{-4}$, así que mi error fue de unos $0{,}32\sigma$, razonable para una sola realización. La versión paralela dio 3,1415469 (78 538 673 aciertos), a unos $0{,}3\sigma$ del valor real.

Reproducibilidad. Con $N = 10^6$ obtuve 785 168 aciertos con p = 1, 2, 4 y 8. Con $N = 10^8$ obtuve 78 538 673 aciertos en las 20 ejecuciones de la tabla (4 valores de p, 5 corridas cada uno). Que el conteo entero sea idéntico es mejor evidencia que comparar decimales: una condición de carrera habría dado conteos distintos.

4. Conexión con mis datos

Usé la mediana porque los tiempos con más hilos varían mucho. Con p = 1 las corridas van de 0,2958 a 0,3003 s (menos de 2 % de rango). Con p = 4 van de 0,1052 a 0,1294 s (cerca de 23 %), por la frecuencia variable y los otros procesos del equipo. Un solo valor atípico movería el promedio.

La mediana con p = 4 fue 0,115829 s, lo que da S = 2,580 y E = 0,645. Con p = 2 la eficiencia fue 0,933, así que el reparto casi no cuesta. La caída a 0,645 y luego a 0,421 con p = 8 (S = 3,364) coincide con mi hardware: tengo 4 núcleos físicos y 8 lógicos. Pasar de 4 a 8 hilos solo aceleró 1,30× (0,115829 → 0,088844 s), porque dos hilos comparten un mismo núcleo. De ahí salió una fracción secuencial efectiva de 18,3 %. Ese valor, que Karp–Flatt reproduce, no es solo código secuencial: absorbe la creación de hilos, la reducción, la memoria y el hiperprocesamiento. Por eso Amdahl predice 1,690 con p = 2 (medí 1,867) y 3,503 con p = 8 (medí 3,364). Con ese f, 64 núcleos darían solo S ≈ 5,10.

5. Reflexión

Aprendí que la corrección de un programa paralelo se demuestra con evidencia concreta, como el conteo idéntico de aciertos, y que más hilos reducen el tiempo pero no la precisión, que depende solo de N. También aprendí que una cláusula mal elegida (dynamic) puede ser 17 veces peor que otra. En un próximo trabajo paralelo mediría desde el principio con una referencia secuencial equivalente, fijaría la afinidad de hilos (OMP_PROC_BIND) para reducir la variabilidad, y probaría p = 6 para ver con más detalle el paso de 4 a 8 hilos.

Bitácora IA:

1. Se ha solicitado a la IA Claude para la generación (vía código) para crear una imagen representativa de la variante acierto y fallo para hacer más fácil la explicación. Revisar Figura 1.

2. Se ha solicitado a la IA Chat GPT para la aclaración de conceptos básicos del método de Monte Carlo, además de haberse generado una explicación útil método de acierto y fallo que no se había quedado del todo claro, lo cual permite comprender el problema para posteriormente explicar las preguntas de la sección 1.

3. Se ha solicitado a la IA gemini para la explicación de la lógica matemática necesaria que requiere un ensayo de Bernoulli con el fin de llegar a la explicación de forma consistente de por qué el error del estimador disminuye en proporción a $1/\sqrt{N}$.

4. Se le preguntó a la IA perplexity que cite las especificaciones de OpenMP (https://www.openmp.org/specifications/) que sirvan de ayuda para responder a la pregunta I2 asociándolo de manera correcta para el caso de acierto y fallo.

5. Se ha pedido a la IA claude para que haga comparativas entre el modelo secuencial y la versión paralela a fin de que se pueda ver ejemplos o demostraciones de si realmente se mantiene el mismo estimador.

6. Se ha solicitado a la IA ChatGPT ayuda para corregir los errores de compilación de la versión secuencial del programa montecarlo_secuencial.c. Se identificó que el código estaba incompleto y se proporcionó una versión corregida que incluye las bibliotecas necesarias, la función main() y la medición del tiempo mediante omp_get_wtime(). Posteriormente, se compiló y ejecutó el programa en WSL.

7. Se ha solicitado a la IA ChatGPT ayuda para analizar los resultados de cinco ejecuciones de la versión secuencial con (N=10^8) muestras. Se calculó el error absoluto de la estimación de (\pi), se explicó su relación con el comportamiento estadístico del método de Monte Carlo y se utilizó la mediana de los tiempos registrados como referencia secuencial (T(1)) para la parte III.

8. Se ha solicitado a la IA ChatGPT orientación para implementar y comprender la versión paralela con OpenMP, incluyendo el uso de omp parallel for, la cláusula reduction(+:dentro) y la selección del número de hilos. También se discutió cómo utilizar un generador determinista basado en el índice de cada muestra y la semilla asignada, con el objetivo de obtener resultados reproducibles al cambiar la cantidad de hilos.

9. Se ha solicitado a la IA ChatGPT una explicación de los problemas que pueden aparecer al utilizar un generador aleatorio compartido entre varios hilos. Se aclararon las posibles condiciones de carrera sobre el estado interno de rand(), la sobrecarga asociada a la sincronización y los inconvenientes estadísticos de inicializar los generadores de todos los hilos con la misma semilla.

10. Se ha solicitado a la IA ChatGPT ayuda para analizar los atributos de compartición de datos de la región paralela de OpenMP. Se revisó el tratamiento de las variables compartidas y privadas, el funcionamiento de la cláusula de reducción y las consecuencias de actualizar el acumulador de aciertos sin dicha cláusula, lo que puede producir una condición de carrera y una estimación incorrecta.

11. Se ha solicitado a la IA ChatGPT orientación para realizar la comparación opcional entre las políticas de planificación schedule(static) y schedule(dynamic). Se explicó cómo crear y compilar dos versiones del programa, manteniendo constantes el número de muestras y la cantidad de hilos, y cómo registrar cinco tiempos por versión para comparar sus medianas.

12. Se ha solicitado a la IA ChatGPT ayuda para interpretar las mediciones obtenidas al comparar las políticas de planificación. A partir de los resultados experimentales, se calculó la mediana de los tiempos de ejecución y se analizó la diferencia de rendimiento entre ambas políticas. Se concluyó que, en las condiciones evaluadas, la planificación estática fue considerablemente más rápida, debido probablemente a la menor sobrecarga de asignación de iteraciones en un problema donde el trabajo por iteración es aproximadamente uniforme.

13. Se ha solicitado a la IA ChatGPT que ayude a organizar y analizar los resultados obtenidos en las ejecuciones del programa paralelo con 1, 2, 4 y 8 hilos. A partir de los tiempos registrados, se calcularon las medianas, el *speedup* y la eficiencia, con el fin de completar la tabla de desempeño de la Parte III.

14. Se ha utilizado la IA ChatGPT para comprender y aplicar la ley de Amdahl en diferentes escenarios, analizando cómo la fracción secuencial limita el rendimiento máximo de un programa paralelo. También se solicitó ayuda para despejar la fracción paralelizable a partir del *speedup* medido con cuatro hilos y contrastar el resultado con la métrica de Karp–Flatt.

15. Se ha solicitado a la IA ChatGPT que explique las diferencias entre el *speedup* ideal, los resultados medidos y las predicciones de la ley de Amdahl. La explicación permitió identificar factores que pueden afectar el rendimiento, como la coordinación de los hilos, la reducción de OpenMP, el acceso a memoria, la caché, las variaciones en la frecuencia del procesador y el hiperprocesamiento.

16. Se ha consultado a la IA ChatGPT para comprender cómo influye el aumento del número de hilos frente al aumento del número de muestras en el método de Monte Carlo. Se aclaró que utilizar más hilos busca reducir el tiempo de ejecución, mientras que aumentar las muestras permite mejorar la precisión estadística de la estimación de π.

17. Finalmente, se solicitó a la IA ChatGPT una explicación de la ley de Gustafson y su diferencia con la ley de Amdahl. Esto permitió comprender por qué el análisis del rendimiento puede cambiar cuando se aumenta el tamaño del problema en lugar de mantenerlo fijo. Las explicaciones y los cálculos se utilizaron como apoyo para desarrollar las respuestas de la Parte III, tomando como referencia las mediciones obtenidas en el equipo.